



ДИСЦИПЛИНА

Разработка приложений на языке Котлин

(полное наименование дисциплины без сокращений)

ИНСТИТУТ

информационных технологий

КАФЕДРА

информационных технологий в атомной энергетике

полное наименование кафедры

ВИД УЧЕБНОГО
МАТЕРИАЛА

Лекция

(в соответствии с пп 1-11)

ПРЕПОДАВАТЕЛЬ

Золотухин Святослав Александрович

(фамилия, имя, отчество)

СЕМЕСТР

5 семестр 2025 – 2026 учебный год

(указать семестр обучения, учебный год)



Лекция 5. Разработка приложений на языке Котлин

ФИО преподавателя: Золотухин Святослав Александрович

e-mail: zolotuhin@mirea.ru



Интерфейсы

```
interface Movable{  
    var speed: Int // объявление свойства  
    fun move()     // определение функции без реализации  
    fun stop(){    // определение функции с реализацией по  
умолчанию  
        println("Остановка")  
    }  
}
```



Интерфейсы

```
class Car : Movable {  
    override var speed = 50  
    override fun move(){  
        println("Машина едет со скоростью $speed км/ч")  
    }  
}
```

```
class Aircraft : Movable{  
    override var speed = 500  
    override fun move(){  
        println("Самолет летит со скоростью $speed км/ч")  
    }  
    override fun stop(){  
        println("Приземление")  
    }  
}
```



Интерфейсы

```
fun main() {  
    val m1: Movable = Car()  
    val m2: Movable = Aircraft()  
    // val m3: Movable = Movable() напрямую объект интерфейса создать нельзя  
  
    m1.move()  
    m1.stop()  
    m2.move()  
    m2.stop()  
}
```

```
Run  TestKt x  
C:\Users\svt\jdk\corretto-17.0.8.1\bin\java.exe "-javaagent:  
Машина едет со скоростью 50 км/ч  
Остановка  
Самолет летит со скоростью 500 км/ч  
Приземление  
Process finished with exit code 0
```



Реализация свойств

```
interface Info{  
    val model: String  
        get() = "Не определено"  
    val number: String  
}  
  
class Car(override val model: String, override var number: String) : Movable, Info{  
    override var speed = 60  
    override fun move(){  
        println("Машина едет со скоростью $speed км/ч")  
    }  
}
```



Правила переопределения

```
open class Music {  
    open fun play() { println("Play music") }  
}
```

```
interface RadioPlayable {  
    fun play() { println("Play radio") }  
}
```

```
class SoundPlayer() : Music(), RadioPlayable {  
    override fun play() {  
        super<Music>.play()    // вызываем Music.play()  
        super<RadioPlayable>.play() // вызываем RadioPlayable.play()  
    }  
}
```



Abstract class VS Interface

Абстрактный класс	Интерфейс
Заготовка для целого семейства классов	Определяет поведение класса или общее поведение для группы независимых друг от друга классов
Нельзя создать экземпляр абстрактного класса	Нельзя создать экземпляр интерфейса
Может содержать как абстрактные, так и конкретные реализации свойств и функций	Может содержать как абстрактные, так и конкретные реализации функций
Класс, который содержит абстрактное свойство или функцию, должен быть объявлен абстрактным	Свойства интерфейсов могут быть абстрактными, а могут иметь get() методы
Может быть без единого абстрактного свойства или функции	Класс может реализовывать несколько интерфейсов
У класса может быть только один суперкласс	Класс должен реализовывать все абстрактные свойства и функции, определённые в интерфейсе
Наследники абстрактного класса должны переопределять все его абстрактные свойства и функции	Если интерфейс реализуется абстрактным классом, то переопределение его абстрактных свойств и функций может быть передано наследникам абстрактного класса
Для того чтобы наследники могли переопределять конкретные реализации свойств и функций, для них в абстрактном классе должен быть явно указан модификатор open	Интерфейс может реализовывать другой интерфейс
У абстрактного класса может быть конструктор	



Вложенные (nested) классы и интерфейсы

```
class User{  
    class Account(val username: String, val password: String){  
        fun showInfo(){  
            println("UserName: $username Password: $password")  
        }  
    }  
}  
  
fun main() {  
    val userAccount = User.Account("User1", "12345678");  
    userAccount.showInfo()  
}
```

Класс Account является вложенным, а класс User - внешним



Вложенные (nested) классы и интерфейсы

```
class User (username: String, password: String){  
    private val account: Account = Account(username, password)  
    private class Account(val username: String, val password: String)  
    fun showAccountInfo(){  
        println("User: ${account.username} Password: ${account.password}")  
    }  
}  
  
fun main() {  
    val andrew = User("User1", "12345678");  
    andrew.showAccountInfo()  
}
```



Вложенные (nested) классы и интерфейсы

```
interface CommunicationProtocol {  
    class ProtocolMessage  
    interface ProtocolHandler  
}
```

```
class DataStorage {  
    class StorageItem  
    interface StorageManager  
}
```



Внутренние (inner) классы

```
class CustomerAccount(private var balance: Int){  
    fun display(){  
        println("Balance = $balance")  
    }  
    class BankTransaction{  
        fun pay(amount: Int){  
            balance -= amount  
            display()  
        }  
    }  
}
```

Вложенные классы в Kotlin не имеют доступа к членам внешнего класса по умолчанию



Внутренние (inner) классы

```
class CustomerAccount(private var balance: Int){  
    fun display(){  
        println("Balance = $balance")  
    }  
    inner class BankTransaction{  
        fun pay(amount: Int){  
            balance -= amount  
            display()  
        }  
    }  
}
```

Класс BankTransaction определен с ключевым словом inner, поэтому имеет полный доступ к свойствам и функциям внешнего класса CustomerAccount.

Однако, если нужно использовать объект такого вложенного класса, то необходимо создать объект внешнего класса.

```
fun main() {  
    val acc1 = CustomerAccount(3000);  
    acc1. BankTransaction().pay(2100)  
}
```



Внутренние (inner) классы

Совпадение имен

this@название_класса.имя_свойства_или_метода

```
class OuterClass {  
    private val N: Int = 1  
    inner class InnerClass {  
        private val N: Int = 1  
        fun action(){  
            println(N)      // N из InnerClass  
            println(this.N)  // N из InnerClass  
            println(this@InnerClass.N) // N из InnerClass  
            println(this@OuterClass.N) // N из OuterClass  
        }  
    }  
}
```



Полиморфизм

```
class MessageService {  
    fun sendMessage(sender: String, receiver: String, message: String) {  
        println("Send a message: \"$message\" from $sender to $receiver")  
    }  
}  
  
fun main() {  
    val messageService = MessageService()  
    messageService.sendMessage("Vasya", "Petya", "Hi!")  
}
```

```
C:\Users\svt\.jdk\corretto-17.0.8.1\bin\java.exe  
Send a message: "Hi!" from Vasya to Petya
```



Полиморфизм

```
class MessageService {  
    fun sendMessage(sender: String, receiver: String, message: String) {  
        println("Send a message: \"$message\" from $sender to $receiver")  
    }  
}  
  
fun main() {  
    val messageService = MessageService()  
    messageService.sendMessage("Vasya", "Petya", "Hi!")  
}
```

```
C:\Users\svt\.jdk\corretto-17.0.8.1\bin\java.exe  
Send a message: "Hi!" from Vasya to Petya
```




Полиморфизм

```
class MessageService {  
    fun sendMessage(sender: String, receiver: String, message: String) {  
        if (sender.contains("@") && receiver.contains("@"))  
            println("Send a message: \"$message\" from $sender to $receiver")  
        }  
    }  
  
    fun main() {  
        val messageService = MessageService()  
        messageService.sendMessage("Vasya@", "Petya@", "Hi!")  
    }  
}
```

```
C:\Users\svt\.jdk\corretto-17.0.8.1\bin\java.exe
```

```
Send a message: "Hi!" from Vasya@ to Petya@
```



Полиморфизм

Полиморфизм по случаю (Ad hoc)

```
class MessageService {  
    fun sendMessage(sender: String, receiver: String, message: String) {  
        if (sender.contains("@") && receiver.contains("@"))  
            println("Send a message: \"$message\" from $sender to $receiver")  
    }  
  
    fun sendMessage(sender: Int, receiver: Int, message: String) {  
        if (sender.toString().length == 9 && receiver.toString().length == 9)  
            println("Send a message: \"$message\" from $sender to $receiver")  
    }  
}
```



Полиморфизм

```
fun main() {  
    val messageService = MessageService()  
  
    messageService.sendMessage("Vasya@", "Petya@", "Hi!")  
    messageService.sendMessage(123456789, 123456789, "Hi from SMS")  
}
```

```
C:\Users\svt\.jdk\corretto-17.0.8.1\bin\java.exe "-javaage  
Send a message: "Hi!" from Vasya@ to Petya@  
Send a message: "Hi from SMS" from 123456789 to 123456789
```



Полиморфизм

Полиморфизм подтипов

```
abstract class MessageService {  
    open fun sendMessage(sender: String, receiver: String, message: String) {  
        println("Send a message: \"$message\" from $sender to $receiver")  
    }  
}  
  
class EmailService: MessageService() {  
    override fun sendMessage(sender: String, receiver: String, message: String) {  
        if (sender.contains("@") && receiver.contains("@"))  
            super.sendMessage(sender, receiver, message)  
    }  
}
```



Полиморфизм

```
class SMSService: MessageService() {  
    override fun sendMessage(sender: String, receiver: String, message: String) {  
        if (sender.length == 9 && receiver.length == 9)  
            super.sendMessage(sender, receiver, message)  
    }  
}  
  
fun main() {  
    val emailService = EmailService()  
    val smsService = SMSService()  
    emailService.sendMessage("Vasya@", "Petya@", "Hi!")  
    smsService.sendMessage("123456789", "123456789", "Hi from SMS")  
}
```

```
C:\Users\svt\.jdk\corretto-17.0.8.1\bin\java.exe "-javaagent:  
Send a message: "Hi!" from Vasya@ to Petya@  
Send a message: "Hi from SMS" from 123456789 to 123456789
```



Анонимные классы и объекты

Анонимные классы не используют ключевое слово **class** для определения. Не имеют имени, но могут наследовать другие классы или применять интерфейсы. Объекты анонимных классов называют **анонимными объектами**.

```
fun main() {  
    val andrew = object {  
        val name = "Andrew"  
        var age = 25  
        fun sayHello(){  
            println("Привет, меня зовут $name")  
        }  
    }  
  
    println("Имя: ${andrew.name} Возраст: ${andrew.age}")  
    andrew.sayHello()  
}
```



Наследование анонимных объектов

```
fun main() {  
    val andrew = object : Person("Андрей"){  
        val company = "JetBrains"  
        override fun sayHello(){  
            println("Привет, меня зовут $name. Я работают в $company")  
        }  
    }  
    andrew.sayHello() // Привет, меня зовут Андрей. Я работают в JetBrains  
}  
  
open class Person(val name: String){  
    open fun sayHello(){  
        println("Привет, меня зовут $name")  
    }  
}
```



Анонимный объект как аргумент функции

```
fun main() {  
    hello(  
        object : Person("Алиса"){  
            val company = "Росатоме"  
            override fun sayHello(){  
                println("Привет, меня зовут $name. Я работают в $company")  
            }  
        })  
}  
  
fun hello(person: Person){  
    person.sayHello()  
}  
  
open class Person(val name: String){  
    open fun sayHello() = println("Привет, меня зовут $name")  
}
```




Анонимный объект как аргумент функции

```
fun main() {  
    val andrew = createPerson("Андрей", "Росатом")  
    andrew.sayHello()  
}  
  
private fun createPerson(_name: String, _company: String) = object{  
    val name = _name  
    val company = _company  
    fun sayHello() = println("Привет, меня зовут $name. Я работают в $company")  
}
```

Если мы хотим иметь доступ к свойствам и методам анонимного объекта, функция, возвращающая этот объект, должна быть объявлена как приватная



Обработка исключений

```
try {  
    // код, генерирующий исключение  
}  
catch (e: Exception) {  
    // обработка исключения  
}  
finally {  
    // постобработка  
}
```



Обработка исключений

```
fun main() {
```

```
    try{
```

```
        val x : Int = 0
```

```
        val z : Int = 0 / x
```

```
        println("z = $z")
```

```
    }
```

```
    catch(e: Exception){
```

```
        println("Exception")
```

```
        println(e.message)
```

```
    }
```

```
}
```

```
C:\Users\svt\.jdk\corretto-17.0.8.1\  
Exception  
/ by zero
```



Обработка исключений

```
try{  
    val x : Int = 0  
    val z : Int = 0 / x  
    println("z = $z")  
}  
catch(e: Exception){  
    println("Exception")  
    println(e.message)  
}  
finally{  
    println("Program has been finished")  
}
```

```
C:\Users\svt\.jdk\corretto-17.0.8.1  
Exception  
/ by zero  
Program has been finished
```



Информация об исключениях

Базовый класс исключений - класс Throwable предоставляет ряд свойств, которые позволяют получить различную информацию об исключении:

- `message`: сообщение об исключении
- `stackTrace`: трассировка стека исключения - набор строк, где было сгенерировано исключение



Информация об исключениях

```
fun main() {  
    try{  
        val x : Int = 0  
        val z : Int = 0 / x  
        println("z = $z")  
    }  
    catch(e: Exception){  
        println(e.message)  
        for(line in e.stackTrace) {  
            println("at $line")  
        }  
    }  
}
```

```
C:\Users\svt\.jdk\corretto-17.0.8  
/ by zero  
at MainKt.main(main.kt:13)  
at MainKt.main(main.kt)
```



Обработка нескольких исключений

```
try {  
    val nums = arrayOf(1, 2, 3, 4)  
    println(nums[6])  
}  
catch(e:ArrayIndexOutOfBoundsException){  
    println("Out of bound of array")  
}  
catch (e: Exception){  
    println(e.message)  
}
```



Оператор throw

```
fun main() {  
    val checkedAge1 = checkAge(5)  
    val checkedAge2 = checkAge(-115)  
}  
  
fun checkAge(age: Int): Int{  
    if(age < 1 || age > 110) throw Exception("Invalid value $age.  
Age must be greater than 0 and less than 110")  
    println("Age $age is valid")  
    return age  
}
```

```
C:\Users\svt\jdk\corretto-17.0.8.1\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Co  
Age 5 is valid  
Exception in thread "main" java.lang.Exception Create breakpoint: Invalid value -115. Age must be greater th  
    at MainKt.checkAge(main.kt:15)  
    at MainKt.main(main.kt:12)  
    at MainKt.main(main.kt)
```




Оператор throw

```
fun main() {  
    try {  
        val checkedAge1 = checkAge(5)  
        val checkedAge2 = checkAge(-115)  
    }  
    catch (e: Exception){  
        println(e.message)  
    }  
}  
  
fun checkAge(age: Int): Int{  
    if(age < 1 || age > 110) throw Exception("Invalid value $age. Age must be greater than 0 and less than 110")  
    println("Age $age is valid")  
    return age  
}
```



Возвращение значения

```
fun main() {  
    val checkedAge1 = try { checkAge(5) } catch (e: Exception) { null }  
    val checkedAge2 = try { checkAge(-125) } catch (e: Exception) { null }  
    println(checkedAge1) // 5  
    println(checkedAge2) // null  
}  
  
fun checkAge(age: Int): Int{  
    if(age < 1 || age > 110) throw Exception("Invalid value $age. Age must be greater than 0 and  
less than 110")  
    println("Age $age is valid")  
    return age  
}
```



Возвращение значения

```
fun main() {  
    val checkedAge2 = try { checkAge(-125) } catch (e: Exception) { println(e.message); 18 }  
    println(checkedAge2)  
}  
  
fun checkAge(age: Int): Int{  
    if(age < 1 || age > 110) throw Exception("Invalid value $age. Age must be greater than 0 and  
less than 110")  
    println("Age $age is valid")  
    return age  
}
```



Спасибо за внимание!